# CMP720 Embedded System Design

WEEK 11

2026 Spring

https://piazza.com/hacettepe.edu.tr/spring2026/cmp720
https://github.com/sepidi/hacettepe-cmp720-spring-2026

# THE PLAN FOR TODAY

**Weeks 8+ Topics: More Optimization**

Precedence Task Graphs, Scheduling Optimization

Another Metaheuristic Approach: Genetic Algorithm

Sequential vs. pipelined execution

# Where We Left Off

**FROM WEEK 10 - Exercise**

***"If tasks had precedence constraints (T2 cannot start until T1 finishes), how would the formulation change?"***

## Assignment problem (Weeks 7-10)

Decided **WHERE** each task runs.

Tasks were *independent* - any task could start at time 0.

Objective: minimize **max load** across processors, minimize energy consumption, etc.

## Scheduling problem (this week)

Tasks have **data dependencies** - a Direct Acyclic Graph (DAG).

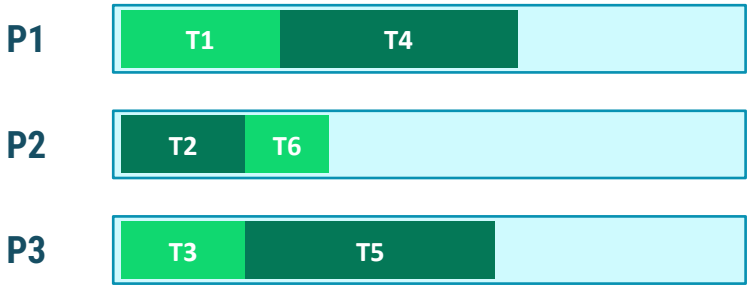
Now we also must decide **WHEN** each task starts.

Objective: minimize **makespan** - finish time of the last task.

# When, Not Just Where

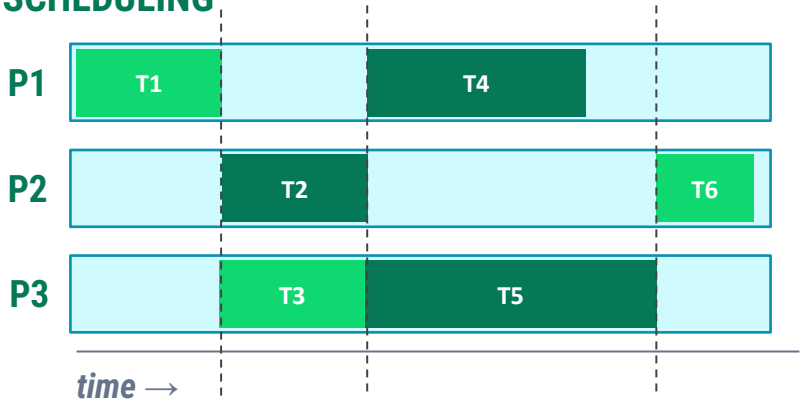
*Two decisions per task instead of one.*

## ASSIGNMENT ONLY




*One decision: x(i,j) - which PE j for task i?*

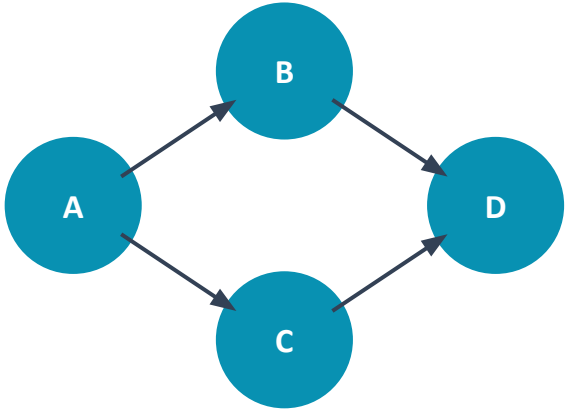
## SCHEDULING




*Two decisions: x(i,j) AND start(i).*

# The Task Graph: A DAG

*Directed Acyclic Graph - the structure that captures "who waits for whom."*




**A is a predecessor of B and C.**
**D has two predecessors.**
**A has no predecessors -> source.**
**D has no successors -> sink.**

### Node (vertex)

A task. Has a known execution time.

### Edge (u -> v)

v cannot start until u finishes.

### Predecessors / successors

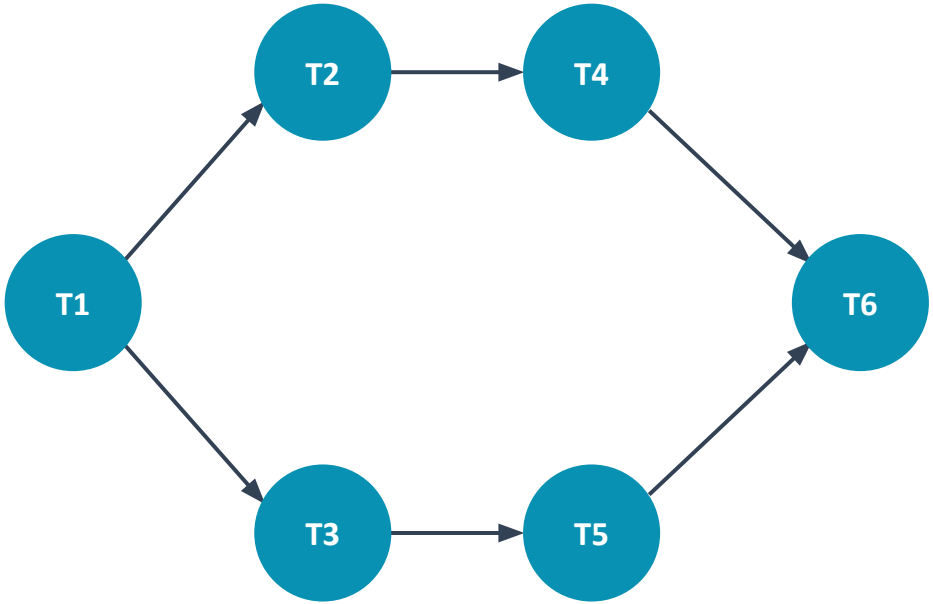
Tasks immediately before / after a node.

### Topological order

Any linear order that respects all edges. Always exists for a DAG.

# Our Running Example

*Six tasks. Diamond structure - two parallel chains from T1 to T6.*




## Execution times

*t(i) = cycles*

| Task | Time |
|---|---|
| T1 | **2** |
| T2 | **4** |
| T3 | **2** |
| T4 | **3** |
| T5 | **2** |
| T6 | **2** |

*Total work = 2 + 4 + 2 + 3 + 2 + 2 = 15 cycles*

# ASAP

## As Soon As Possible Scheduling

Forward sweep through the DAG. Every task starts the moment its predecessors are done.

# ASAP: As Soon As Possible

## Definition

Each task starts the instant all its predecessors have finished.

**Formally:**

$$\mathbf{ASAP(v)} = \max\ (\ ASAP(u) + t(u)\ )$$
*over u in pred(v)*

Source nodes (no predecessors) start at **time 0**.

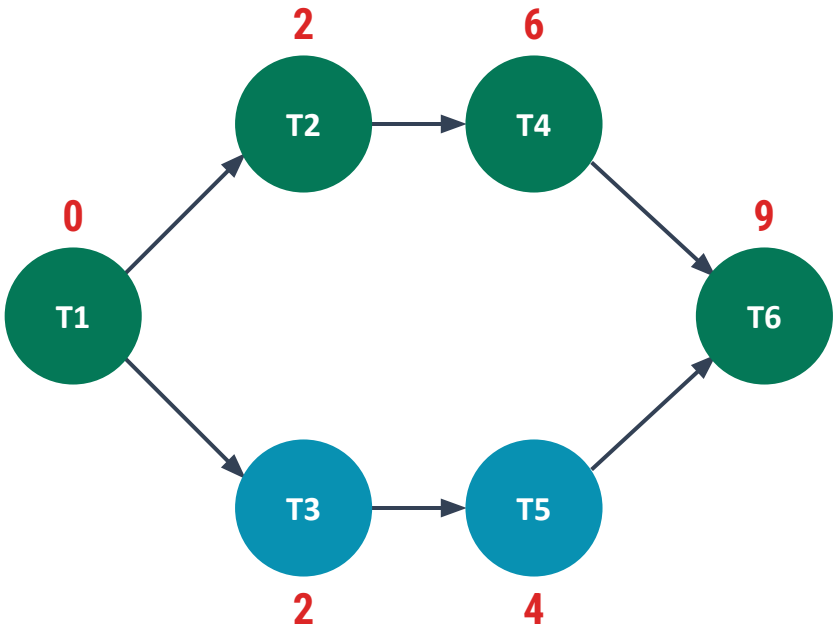
***Assumes unbounded resources - every ready task runs immediately.***

## Algorithm

```
compute topological order of V

for v in topological order:
    if pred(v) is empty:
        ASAP(v) = 0
    else:
        ASAP(v) = max over u in pred(v)
                  of (ASAP(u) + t(u))

makespan = max over v of (ASAP(v) + t(v))
```

***Single forward pass. Linear time in nodes + edges.***

# ASAP on Our Example




*ASAP start times shown in red. Green nodes form the critical path.*

**Execution times**

*t(i) = cycles*

| Task | Time |
|---|---|
| T1 | 2 |
| T2 | 4 |
| T3 | 2 |
| T4 | 3 |
| T5 | 2 |
| T6 | 2 |

## Trace

| Task | Calculation | ASAP |
|---|---|---|
| **T1** | *no preds* | **0** |
| **T2** | 0 + 2 | **2** |
| **T3** | 0 + 2 | **2** |
| **T4** | 2 + 4 | **6** |
| **T5** | 2 + 2 | **4** |
| **T6** | max(6+3, 4+2) | **9** |

**Makespan = 11** *(= ASAP(T6) + t(T6) = 9 + 2)*

# ASAP gives the Critical Path - and a Lower Bound

## Critical Path

The longest path of work from any source to any sink. On our DAG: T1 → T2 → T4 → T6 with length 11.

## Lower Bound

No schedule can finish before the critical path length, no matter how many PEs you have.

## Why it matters

When list scheduling later achieves makespan = CP, we know it found the optimum.

**Two lower bounds: critical path (CP) and total-work / num-PEs.**
**Optimal makespan ≥ max of the two.**

# ALAP

## As Late As Possible Scheduling

Backward sweep from a deadline. Every task starts the latest possible without delaying its successors.

# ALAP: As Late As Possible

## Definition

**Pick a deadline D - typically the ASAP makespan.**

**Each task starts as late as possible without pushing any successor past D.**

$$\mathbf{ALAP(v)} =$$
$$D - t(v) \quad \textit{if v is a sink}$$
$$\min\,(ALAP(w) - t(v)) \quad \textit{over w in succ(v)}$$

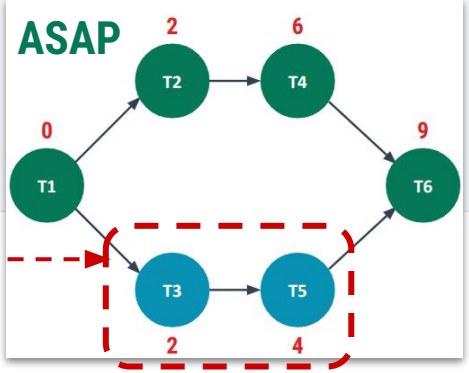
*Single backward pass through reverse topological order.*

## Algorithm

```
compute reverse topological order of V

makespan = given latency constraint

for v in reverse topological order:
    if succ(v) is empty:
        ALAP(v) = makespan - t(v)
    else:
        ALAP(v) = min over u in succ(v)
                  of (ALAP(u) - t(v))
```

*Same makespan as ASAP - by construction. What changes are the start times.*

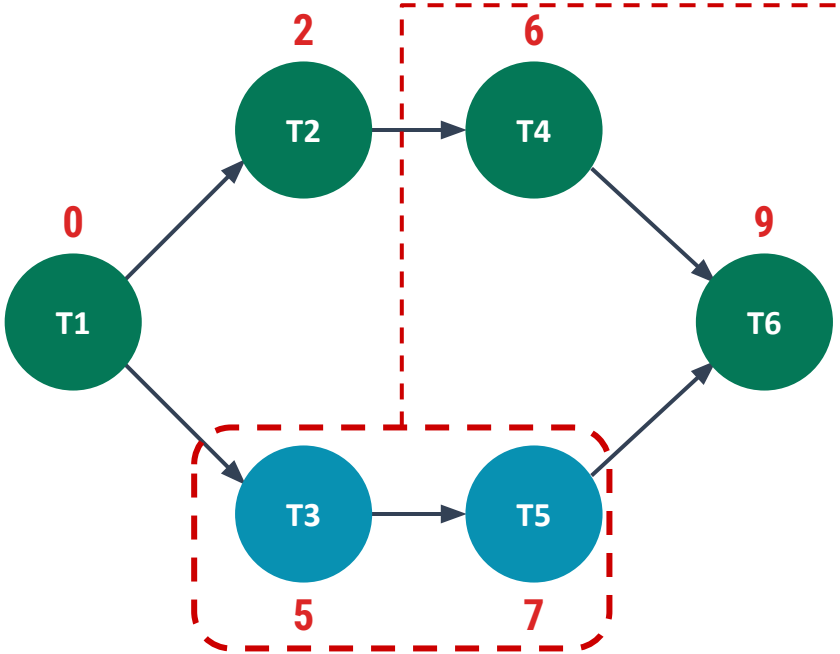
# ALAP on Our Example




**Execution times**

*t(i) = cycles*

| Task | Time |
|---|---|
| T1 | 2 |
| T2 | 4 |
| T3 | 2 |
| T4 | 3 |
| T5 | 2 |
| T6 | 2 |



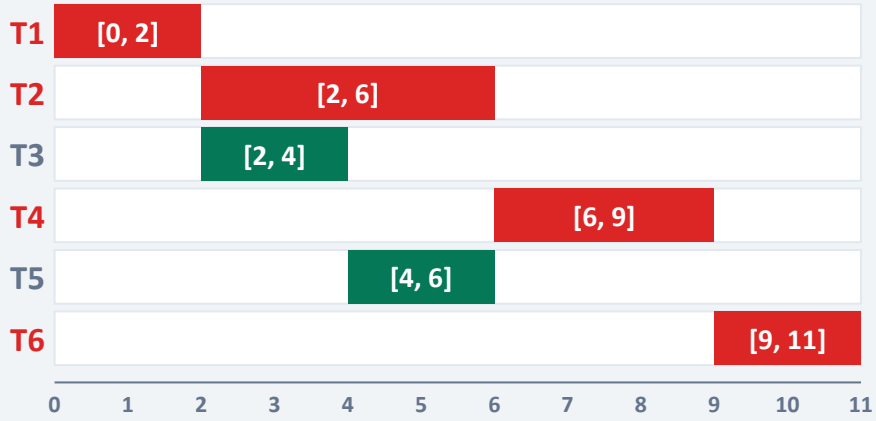

*ALAP start times shown in red. Note the change in the start times of the nodes NOT on the critical path.*

## Trace (D = 11, calculated by ASAP)

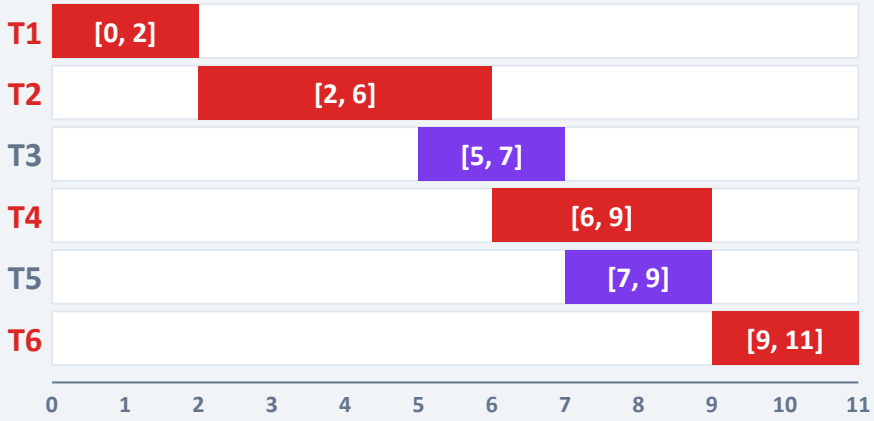
| Task | Calculation | ALAP |
|---|---|---|
| **T6** | 11 - 2 (sink) | 9 |
| **T5** | 9 - 2 | 7 |
| **T4** | 9 - 3 | 6 |
| **T3** | 7 - 2 | 5 |
| **T2** | 6 - 4 | 2 |
| **T1** | min(2-2, 5-2) | 0 |

# ASAP vs ALAP - Same Makespan, Different Timing

## ASAP - run as soon as inputs are ready




## ALAP - delay until just before the deadline




### Notice

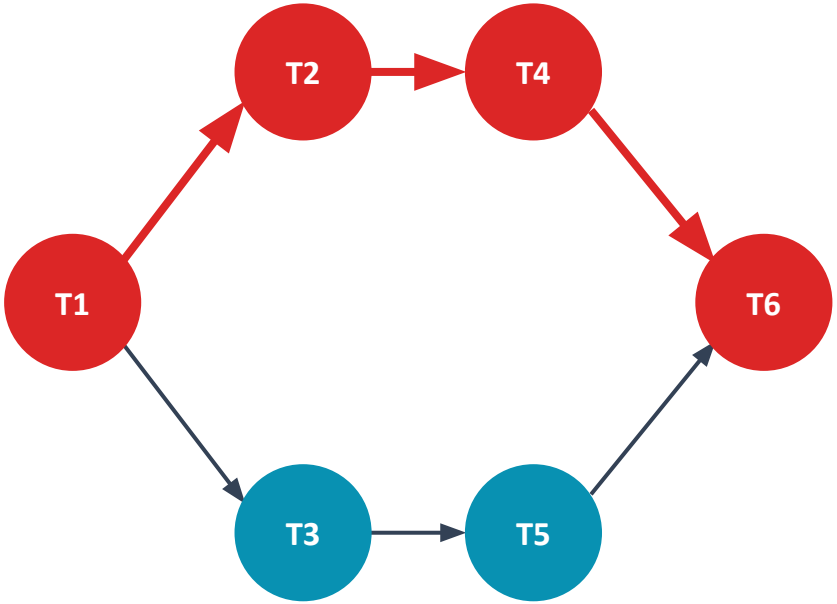
**Critical path tasks (red) sit in identical positions in both charts. T3 shifts from [2,4] to [5,7]. T5 shifts from [4,6] to [7,9]. The free tasks soak up their slack.**

# Slack

## How much wiggle room does each task have?

Slack(v) = ALAP(v) - ASAP(v)
Zero slack means
a task is on the critical path.

# Slack and the Critical Path




*Critical path (slack = 0): T1 → T2 → T4 → T6*

## Slack table

| Task | ASAP | ALAP | Slack |
|---|---|---|---|
| T1 | 0 | 0 | 0 |
| T2 | 2 | 2 | 0 |
| T3 | 2 | 5 | 3 |
| T4 | 6 | 6 | 0 |
| T5 | 4 | 7 | 3 |
| T6 | 9 | 9 | 0 |

**Slack is a priority signal**

**Smaller slack = more urgent.**

**List scheduling** will use this.

# List Scheduling

## Bringing resources back into the picture

ASAP / ALAP assumed unbounded resources. With only *k* PEs, we need a smart heuristic.

# What If We Only Have *k* PEs?

## Unlimited PEs (ASAP)

Every ready task starts immediately.

Makespan = critical path length.

On our example: makespan = **11**, needs up to **3 PEs** at peak.

## Only *k* PEs (*k* = 2 for example)

When more tasks are ready than PEs, who waits?

Deciding the order is now a real decision.

**Optimal scheduling under resource constraints is NP-hard**. We use a heuristic: **list scheduling**.

## Two lower bounds on optimal makespan:

```
Critical path: 11    |    Work bound: ceil(15 / 2) = 8    |
Combined: makespan ≥ max(11, 8) = 11
```

# List Scheduling: The Algorithm

*Greedy. Event-driven. Picks tasks by priority when PEs free up.*

## Algorithm

```
t = 0
ready = sources of the DAG

while not all tasks finished:
    sort ready by priority
    while there is a free PE and a ready task:
        v = highest-priority ready task
        assign v to a free PE, start at time t

    advance t to next finish event
    update ready set with newly enabled tasks
```

### Ready set

Tasks whose predecessors have all finished but haven't started yet.

### Priority function

Determines who gets the next free PE. Several reasonable choices.

### No backtracking

Once a task starts, it runs to completion. **Greedy = fast but not optimal.**

# Priority Functions

*Different priorities push the scheduler toward different schedules.*

### Smallest slack first

Most urgent tasks first. Critical-path tasks get the PE before anyone else.

*Best general-purpose default.*

### Longest remaining path (HLP)

Pick the task whose downstream chain to the sink is longest.

*Tries to keep the bottleneck busy.*

### Smallest level / topological depth

Pick tasks earliest in the topological order first.

*Simple, but ignores execution times.*

### Largest task time first

Schedule big tasks early so they don't dangle near the deadline.

*Common in classical job-shop heuristics.*

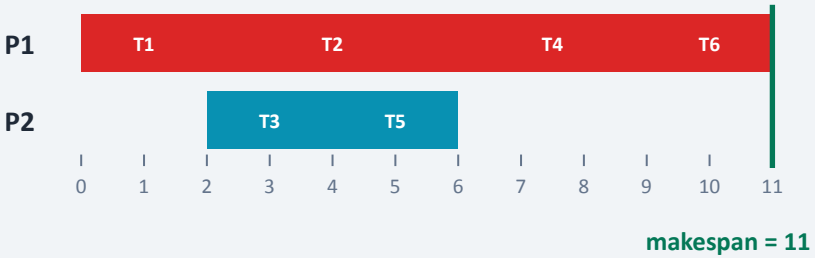
***On larger instances, priority choice can produce noticeably different makespans. Try several, pick the best.***

# Worked Example: Smallest-Slack-First on 2 PEs

## Trace

| t | Ready | Pick | Assignment |
|---|---|---|---|
| **0** | T1 | T1 (slack 0) | T1 → P1, ends at 2 |
| **2** | T2(0), T3(3) | T2 first, then T3 | T2 → P1 (ends 6), T3 → P2 (ends 4) |
| **4** | T5(3) | T5 | T5 → P2, ends at 6 |
| **6** | T4(0) | T4 | T4 → P1, ends at 9 |
| **9** | T6(0) | T6 | T6 → P1, ends at 11 |

## Resulting schedule




### Result

**Makespan = 11 = critical path length.** On this instance, list scheduling matched the lower bound - it's optimal.

P2 sits idle from time 6 to 11. The critical path constrains us, not the resources. *The work bound (8) is irrelevant here.*

# List Scheduling Is a Heuristic

*It can be optimal. It can also be off. The priority function is the lever.*

### When CP ≥ work bound

Like our example. Smallest-slack-first hits the critical path - which is the lower bound. Heuristic = optimal.

**Optimal**

### When the bottleneck is resources

Many short, parallelizable tasks competing for few PEs. Different priorities give different schedules; some near the work bound, some worse.

**Try several**

### Pathological cases

Adding a PE or shortening a task can make list-scheduling makespan worse. Greed has limits.

**ILP if it fits**

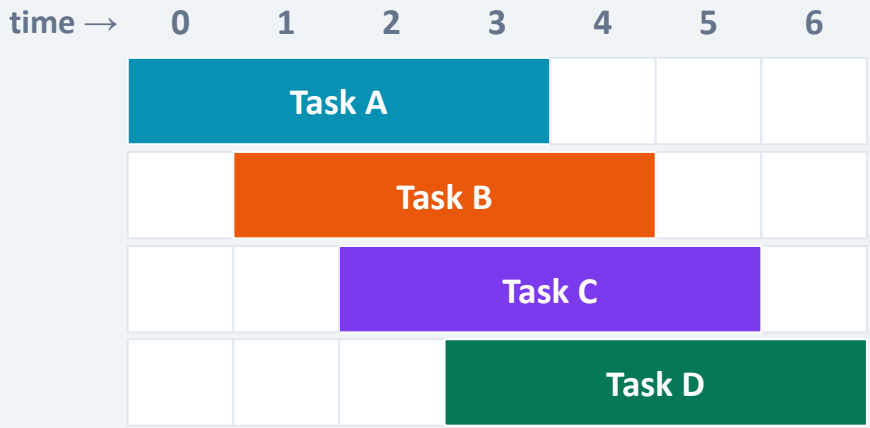
*In practice: run list scheduling with 2-3 priorities, take the best.*

# On Pipelined Execution

Functional Units (FUs) with internal pipeline stages can host multiple operations at once - new task in, old task still in flight.

# Latency vs. Initiation Interval

*Example of a pipelined execution of tasks on a FU.*




*Task A enters at t=0, advances each cycle, exits at t=4, Task B can enter at t=1 at the same FU because II-1.*

## Latency (L)

Cycles from input arrival to output ready. Single-op delay.
*In our diagram: L = 4.*

## Initiation Interval (II_unit)

Min cycles between successive op starts on the same FU.

Pipelined: II_unit = 1 (typical, like above).
Non-pipelined: II_unit = L.

## Typical HLS functional units

```
E.g., pipelined integer adder: L=4, II=1.
```

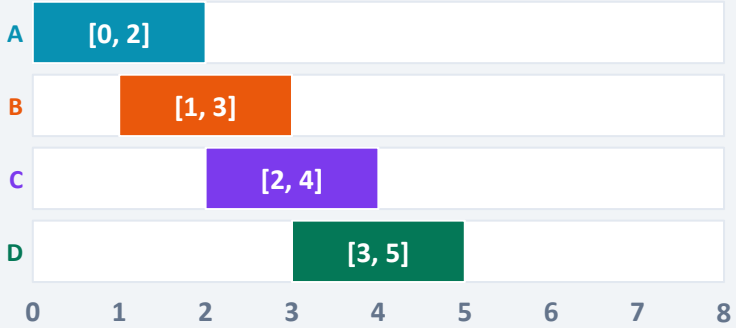
# Effect on Scheduling: One Adder, Four Adds

*Same four independent operations, same FU latency. Pipelining changes when each one can start.*

## Non-pipelined adder (II_unit = L = 2)

**Makespan = 8**

## Pipelined adder (II_unit = 1, L = 2)




**Makespan = 5**

### Scheduling rule for pipelined FUs

`PE next-free = task.start + II_unit` *(not `task.finish`).*

Plug into list scheduling, ILP, or SA without changing the algorithm.

# Adding Extra Constraints

**Task-PE compatibility: not every task can run on every PE**

Real heterogeneous platforms have specialized PEs - DSPs, accelerators, secure cores. Some tasks are restricted.

# Task-PE Compatibility

*A binary matrix says which tasks are allowed on which PEs.*

## Why?

Real embedded platforms are often heterogeneous:

- FFT only on the DSP core
- Crypto only on the secure enclave
- Conv layers only on the NPU
- Floating-point only on cores with FPU

*Forces some tasks onto specific PEs - and shapes the schedule.*

### Example matrix (1 = allowed)

| compat(i,j) | P1 (CPU) | P2 (CPU) | P3 (DSP) |
|---|---|---|---|
| T1 | 1 | 1 | 1 |
| T2 | 1 | 1 | 0 |
| T3 (FFT) | 0 | 0 | 1 |
| T4 | 1 | 1 | 0 |
| T5 (FFT) | 0 | 0 | 1 |
| T6 | 1 | 1 | 1 |

*Here: T3 and T5 are FFT tasks - they must run on the DSP. T2 and T4 cannot run on the DSP because they require control-heavy or system-level capabilities*

# Compatibility in Each Method

*Same idea everywhere: filter the choices to the compatible PEs.*

## ILP

*Add a single linear constraint*

```
x(i,j) ≤ compat(i,j)
```

Forbids assigning task i to PE j when compat(i,j) = 0. Solver handles the rest.

## List scheduling

*Filter at PE selection*

```
free ∩
{ j : compat(i,j) = 1 }
```

When picking a PE for ready task i, restrict to compatible PEs only. If none free, task waits.

## Simulated annealing

*Restrict the neighborhood*

```
j' ~ Uniform({j :
compat(i,j) = 1})
```

When proposing a reassignment for task i, sample only from its compatible PE set. Infeasible moves never happen.

*One concept, three plug-in points. The optimization machinery doesn't change.*

# Extended Problem Formulation

## Problem statement

We are given a directed acyclic task graph and a set of processing elements. Each task has an execution time that may depend on the PE running it, and not every task is allowed to run on every PE. Time is discretised into integer slots $0, 1, \ldots, T-1$. We want to assign each task to a PE *and* pick a start slot so that:

- every precedence edge is respected,
- no PE runs two tasks at the same time,
- every task runs on a compatible PE,
- per-PE memory limits are not exceeded,
- the overall makespan (latest completion time) is minimised.

# Extended Problem Formulation (cont.)

## 1 Sets and parameters

| | |
|---|---|
| $V = \{1, \ldots, n\}$ | set of tasks (vertices of the DAG) |
| $E \subseteq V \times V$ | set of precedence edges; $(u, v) \in E$ means $u$ must finish before $v$ starts |
| $M = \{1, \ldots, k\}$ | set of processing elements |
| $\mathcal{T} = \{0, 1, \ldots, T-1\}$ | set of integer time slots |
| $\mathrm{exec}_{i,j} \in \mathbb{Z}_{>0}$ | execution time of task $i$ on PE $j$ |
| $\mathrm{mem}_i \in \mathbb{Z}_{>0}$ | memory footprint of task $i$ |
| $\mathrm{cap}_j \in \mathbb{Z}_{>0}$ | memory capacity of PE $j$ |
| $c_{i,j} \in \{0, 1\}$ | compatibility flag: $c_{i,j} = 1$ iff task $i$ may run on PE $j$ |
| $T$ | time horizon, an upper bound on the makespan (e.g. $T = \sum_i \max_j \mathrm{exec}_{i,j}$) |

If your platform is homogeneous, set $\mathrm{exec}_{i,j} \equiv \mathrm{exec}_i$ and ignore the $j$ subscript. If compatibility is unrestricted, set $c_{i,j} \equiv 1$.

# Extended Problem Formulation (cont.)

## 2 Decision variables

| | |
|---|---|
| $x_{i,j,t} \in \{0,1\}$ | $x_{i,j,t} = 1$ iff task $i$ *starts* at time $t$ on PE $j$ |
| $C_{\max} \in \mathbb{R}_{\geq 0}$ | makespan |

Week 10 had $x_{i,j} \in \{0,1\}$. Add a time dimension:

$$x_{i,j,t} \in \{0,1\}, \quad x_{i,j,t} = 1 \iff \text{task } i \textit{ starts} \text{ at time } t \text{ on processor } j.$$

There are no separate start-time variables. Two linear helper expressions in $x$ recover the start and end times we need for the precedence and makespan rows:

$$\text{start}(i) = \sum_{j \in M} \sum_{t \in \mathcal{T}} t \cdot x_{i,j,t}, \qquad \text{end}(i) = \sum_{j \in M} \sum_{t \in \mathcal{T}} \left(t + \text{exec}_{i,j}\right) x_{i,j,t}.$$

# Extended Problem Formulation (cont.)

## 3 Objective

$$\min \quad C_{\max} \tag{1}$$

## 4 Constraints

### (C1) Each task is started exactly once

$$\sum_{j \in M} \sum_{t \in \mathcal{T}} x_{i,j,t} \;=\; 1 \qquad \forall\, i \in V \tag{2}$$

Week 10's assignment constraint with one extra summation over $t$.

### (C2) Memory capacity

$$\sum_{i \in V} \mathrm{mem}_i \cdot \Big( \sum_{t \in \mathcal{T}} x_{i,j,t} \Big) \;\leq\; \mathrm{cap}_j \qquad \forall\, j \in M \tag{3}$$

The inner sum collapses to "is task $i$ on PE $j$?" — so this is Week 10's memory row verbatim.

# Extended Problem Formulation (cont.)

**(C3) Compatibility**

$$x_{i,j,t} \;\le\; c_{i,j} \qquad \forall\, i \in V,\; j \in M,\; t \in \mathcal{T} \tag{4}$$

Week 10's compatibility constraint with one extra $\forall t$. In code we typically *eliminate* the variable rather than constrain it: we never create $x_{i,j,t}$ when $c_{i,j} = 0$, which gives the same feasible region with fewer LP rows.

**(C4) Precedence: $\mathbf{end}(u) \le \mathbf{start}(v)$ for every edge $(u, v)$**

$$\sum_{j \in M} \sum_{t \in \mathcal{T}} \left(t + \mathrm{exec}_{u,j}\right) x_{u,j,t} \;\le\; \sum_{j \in M} \sum_{t \in \mathcal{T}} t \cdot x_{v,j,t} \qquad \forall\, (u, v) \in E \tag{5}$$

**(C5) No two tasks active on the same PE at the same slot**

$$\sum_{i \in V} \sum_{\tau = \max(0,\, t - \mathrm{exec}_{i,j} + 1)}^{\min(t,\, T - \mathrm{exec}_{i,j})} x_{i,j,\tau} \;\le\; 1 \qquad \forall\, j \in M,\; \forall\, t \in \mathcal{T} \tag{6}$$

# Extended Problem Formulation (cont.)

**(C6) Makespan dominates every task end**

$$C_{\max} \geq \sum_{j \in M} \sum_{t \in \mathcal{T}} \left(t + \mathrm{exec}_{i,j}\right) x_{i,j,t} \qquad \forall\, i \in V \tag{7}$$

**(C7) Variable domains**

$$x_{i,j,t} \in \{0, 1\}, \qquad C_{\max} \geq 0 \tag{8}$$

# Extended Problem Formulation (cont.)

## 5 Model size

For $n$ tasks, $k$ PEs, time horizon $T$, and $|E|$ precedence edges:

| | **count** | **type** |
|---|---|---|
| $x_{i,j,t}$ | $nkT$ (or fewer with variable elimination) | binary |
| $C_{\max}$ | 1 | continuous |
| (C1) | $n$ | equality |
| (C2) | $k$ | inequality |
| (C3) | $nkT$ (or 0 when variables are dropped) | inequality |
| (C4) | $\|E\|$ | inequality |
| (C5) | $kT$ | inequality |
| (C6) | $n$ | inequality |

The variable count grows linearly with $T$, so this formulation is best suited to instances where the makespan stays modest (slots in the dozens to a few hundred). For the running example below ($n = 6$, $k = 3$, $T = 15$): roughly 270 binaries before elimination, and the no-overlap rows total $kT = 45$. Modern solvers handle this in milliseconds.

# Exercises (Scheduling)

*Use the same DAG with t = (2, 4, 2, 3, 2, 2). Two PEs unless stated otherwise.*

1. **ASAP / ALAP / slack**
   Reproduce the trace tables for ASAP, ALAP (D = 11), and slack. Identify the critical path.

2. **List scheduling, alternative priority**
   Run list scheduling on 2 PEs using "largest task time first" instead of smallest slack. Compare the makespan.

3. **Reduce to 1 PE**
   Find the optimal makespan with a single PE. Why is it independent of the priority function?

4. **Add task-PE compatibility**
   Set compat(T3,P1) = compat(T3,P2) = 0 (so T3 must run on P3, a third PE). Rerun list scheduling on 3 PEs - does the makespan change? Could II shrink?

5. **Pipelined feasibility**
   Try to construct a modulo schedule with II = 8 on 2 PEs. If you cannot fit it, what's the smallest II you can achieve and why?

# What's Next

*We have ILP, SA, and now scheduling under precedence. One more big tool to add.*

## Next: Genetic Algorithms

### Population, not a single point

Many candidate schedules evolved in parallel - very different from SA's single-walker search.

### Crossover

Recombine two parent assignments y_a and y_b into a child. Recombination is the key operator GAs add.

### Same encoding

y = [1,2,3,1,3,2] - the chromosome is the same vector we used in SA. Plug it into a new search engine.

**After GA: combining scheduling + assignment + metaheuristics. Same problem class, richer toolbox.**

# Why a Second Metaheuristic?

*SA is one search style. GA is another. Different inductive biases, different strengths.*

### Population, not a single point

SA explores with one walker. GA carries many candidate schedules through evolution and lets them recombine. Larger problems benefit from parallel exploration.

### Recombination, not just perturbation

SA only mutates. GA also performs crossover - it splices two good parents together. This combines partial solutions from different regions of the search space.

### A foundation for more

GA opens the door to NSGA-II for multi-objective scheduling, CMA-ES for continuous problems, and Memetic Algorithms (GA + local search) - the workhorse of HLS optimization.

# Genetic Algorithm

A Different Metaheuristics Approach

# The Five GA Ingredients

*Anything we call a Genetic Algorithm has these five parts. Different choices, different GAs.*

### 1 Encoding

How a candidate solution is represented as a chromosome (gene string).

### 2 Fitness function

A number that says how good a chromosome is. Lower = better for makespan.

### 3 Selection

Pick parents biased toward fitter individuals. Tournament, roulette, rank.

### 4 Crossover

Combine two parents into children. One-point, uniform, problem-specific.

### 5 Mutation

Random small changes to a child. Maintains diversity, escapes plateaus.

### + Plus a few hyperparameters

Population size, mutation rate, crossover rate, elitism count, stopping criterion.

# Anatomy of a Generation

*One outer loop. Each iteration replaces the population.*

**Algorithm**

```
P = random population of N chromosomes
evaluate fitness of each in P

while not done:
    sort P by fitness
    P' = top elite of P
    while |P'| < N:
        p1, p2 = select two parents
        c1, c2 = crossover(p1, p2)
        c1 = mutate(c1); c2 = mutate(c2)
        add c1, c2 to P'
    P = P'

return best individual in P
```

**Generation flow**

# Encoding: The Chromosome

*A vector of length 6: one gene per task, value = assigned PE.*

| T1 | T2 | T3 | T4 | T5 | T6 |
|---|---|---|---|---|---|
| 1 | 2 | 3 | 1 | 3 | 2 |

*PE assigned (gene value)*

1 - PE 1 | 2 - PE 2 | 3 - PE 3 ...

## Reading this chromosome

T1 → P1, T2 → P2, T3 → P3, T4 → P1, T5 → P3, T6 → P2

*All three PEs are used. Diamond split across PEs.*

## Search space

**k possible PEs × n tasks → $k^n$ chromosomes**

*$3^6 = 729$ here. For 50 tasks on 8 PEs: $8^{50} \approx 10^{45}$ - way too big to enumerate.*

# Decoder: Chromosome → Schedule → Fitness Function

*The chromosome only says WHERE. The decoder figures out WHEN, respecting the DAG.*

## Chromosome

| T1 | T2 | T3 | T4 | T5 | T6 |
|---|---|---|---|---|---|
| 1 | 2 | 3 | 1 | 3 | 2 |

y = [1, 2, 3, 1, 3, 2]

## Inputs

- Chromosome y
- Task graph (DAG)
- Execution times t(i)

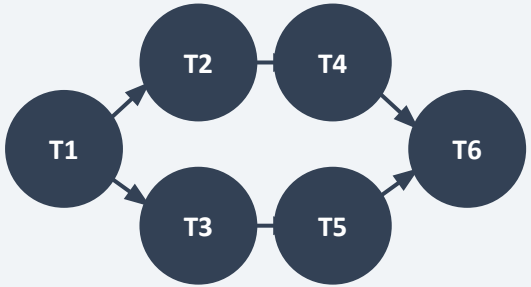
decode →

## Decoder

```
finish = {}   # finish[u] = when task u ended
pe_free = {}  # pe_free[j] = when PE j is next free

# parents before children
for v in topological order:
    pe = y[v]  # which PE for v?

    # Wait 1: data dependencies
    pred_done = max(finish[u] for u in pred(v))

    # Wait 2: PE availability
    start[v] = max(pred_done, pe_free[pe])

    # Run v on its PE
    finish[v] = start[v] + t(v)
    pe_free[pe] = finish[v]  # PE busy till v ends

makespan = max(finish[v] for all v)
```

# Decoder Walk-Through: y = [1, 1, 2, 1, 2, 1]

| T1 | T2 | T3 | T4 | T5 | T6 |
|---|---|---|---|---|---|
| 1 | 1 | 2 | 1 | 2 | 1 |



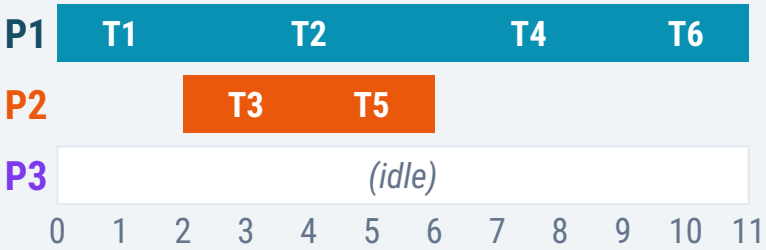

**Fitness function!**

Makespan

**11**

| Task | PE (y) | Pred. done | PE free | Start | Finish |
|---|---|---|---|---|---|
| T1 | P1 | 0 | 0 | 0 | 2 |
| T2 | P1 | 2 | 2 | 2 | 6 |
| T3 | P2 | 2 | 0 | 2 | 4 |
| T4 | P1 | 6 | 6 | 6 | 9 |
| T5 | P2 | 4 | 4 | 4 | 6 |
| T6 | P1 | 9 | 9 | 9 | 11 |

## Schedule




*This chromosome doesn't use P3 - it's still a valid (and optimal) schedule. The decoder doesn't insist on using all PEs.*

# Selection: Pick Parents Biased Toward Fitness

*Best, but not deterministic. We want some chance for weaker individuals to contribute.*

### Tournament (k = 2)

Pick k random individuals. Return the fittest. Repeat for each parent slot.

*Most common. Tunable pressure: larger k = stronger preference for the best.*

**OUR DEFAULT**

### Roulette wheel

Probability of selection proportional to fitness (or 1/fitness for minimization).

*Classical, but unstable when fitness range is wide.*

### Rank selection

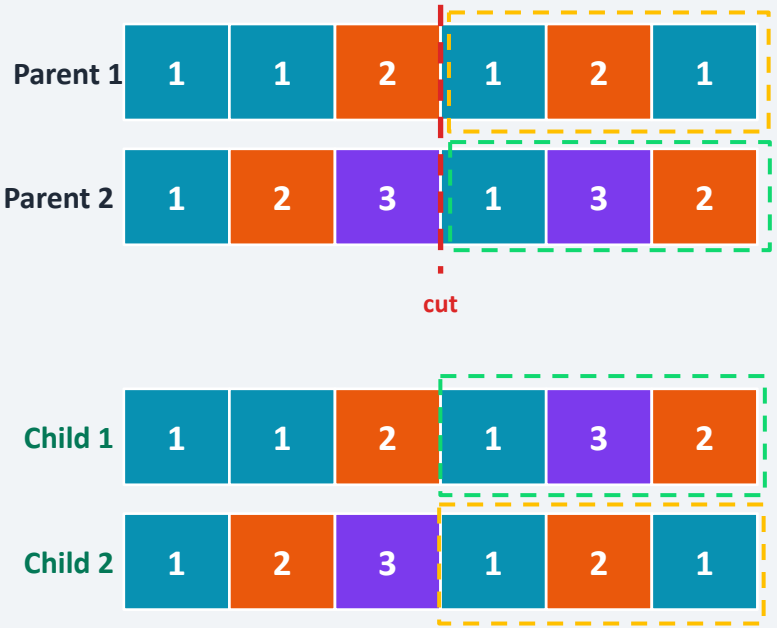
Sort by fitness, sample with probability based on rank, not raw value.

*More stable than roulette. Robust to fitness scaling.*

*Higher pressure (larger tournament k) - converges faster but loses diversity quickly.*

# Crossover: Recombine Two Parents

*Cut the chromosomes, swap the tails - the children inherit pieces of both.*




## One-point crossover

Choose a cut point uniformly at random in {1, ..., n-1}.

**Child 1 = parent1[:k] + parent2[k:]**
**Child 2 = parent2[:k] + parent1[k:]**

*Variants: **two-point** (two cuts, swap middle), **uniform** (each gene independently from one parent).*

***Crossover happens with probability p_cx (typically 0.6-0.9). With probability (1-p_cx), children are exact copies.***

# Crossover: Recombine Two Parents

| | |
|---|---|
| Chromosome1 | 11011\|00100\|110110 |
| Chromosome2 | 10101\|11000\|011110 |
| Offspring1 | 11011\|11000\|110110 |
| Offspring2 | 10101\|00100\|011110 |

Two Point Crossover

Parent :

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1

Children :

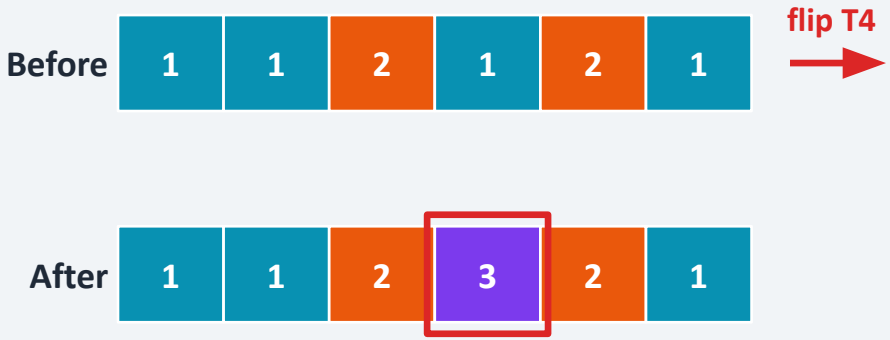
1 0 0 0 1 1 0 1 0 1 0 0 1 0 0 1 1 1 1 0 1

0 1 1 1 0 0 1 0 1 0 1 1 0 1 1 0 0 0 0 1 0

Uniform Crossover

# Mutation: Random Small Changes

*Each gene independently mutates with low probability. Maintains diversity, escapes plateaus.*




## Mutation rate

**p_mut = 0.05 - 0.20** typical.

Per-gene probability of being replaced with a uniformly random PE.

### p_mut too low

Population converges to a single chromosome, GA stalls. Crossover alone can't reach genes that are extinct everywhere.

### p_mut too high

GA degenerates into random search. Good genes get destroyed faster than selection can preserve them.

# Worked Example

# Generation 0: Initial Population (N = 4)

*Four random chromosomes (small for the demo - real GAs use 50-200).*

| | T1 | T2 | T3 | T4 | T5 | T6 | Makespan |
|---|---|---|---|---|---|---|---|
| y_A | 1 | 1 | 2 | 1 | 2 | 1 | 11 |
| y_B | 1 | 2 | 3 | 1 | 3 | 2 | 11 |
| y_C | 1 | 1 | 1 | 2 | 2 | 3 | 13 |
| y_D | 1 | 1 | 1 | 1 | 1 | 1 | 15 |

**Population stats**

Best: 11 *(y_A, y_B)*
Worst: 15 *(y_D)*
Average: 12.5

**Notice**

Two individuals (y_A, y_B) already hit the optimum 11 by chance. The other two are slack. The GA's job is now to spread good genes across the whole population.

# Step 1: Tournament Selection

*Need 4 parent slots (one for each child). Run 4 tournaments of size 2.*

| Round | Contestants | Makespans | Winner |
|---|---|---|---|
| 1 | y_A vs y_C | 11 vs 13 | y_A |
| 2 | y_B vs y_D | 11 vs 15 | y_B |
| 3 | y_A vs y_B | 11 vs 11 | y_A |
| 4 | y_C vs y_D | 13 vs 15 | y_C |

**Selected parents**

y_A
y_B
y_A
y_C

*Pair them up:*

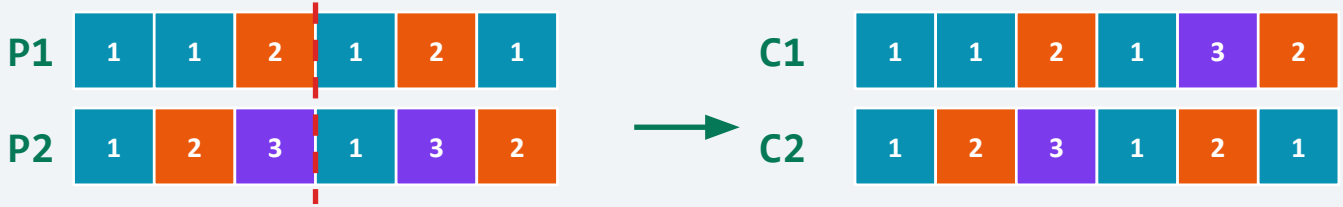
(y_A, y_B)
(y_A, y_C)

*y_D (the worst) was never picked. Tournament selection naturally weeds out the weakest.*

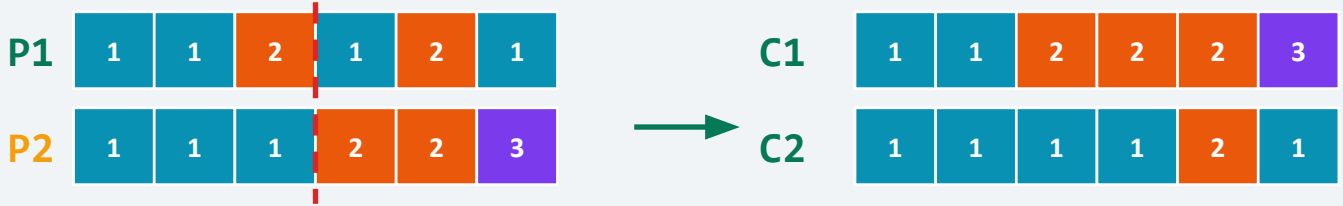
# Step 2: Crossover

*Cut at position 3 for both pairs. Tails are swapped.*

**Pair 1: (y_A, y_B)**




**Pair 2: (y_A, y_C)**

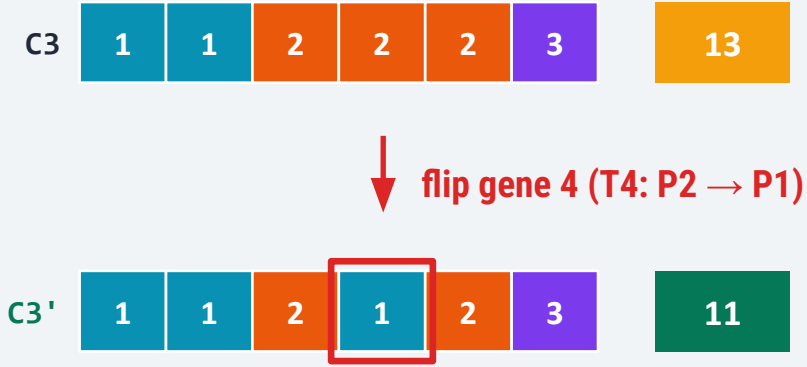



**Children of generation 0:**

```
C1=[1,1,2,1,3,2] m=11   |   C2=[1,2,3,1,2,1] m=11   |
C3=[1,1,2,2,2,3] m=13   |   C4=[1,1,1,1,2,1] m=13
```

# Step 3: Mutation

*Flip each gene with probability p_mut = 0.1. Most stay put; one lucky flip improves C3.*




## Mutation is random

This time we got lucky - the flip improved fitness.

More often, mutation is neutral or harmful. Selection in the next generation will weed out the harmful ones.

**Children after mutation:**

| | | | |
|---|---|---|---|
| C1 | = [1,1,2,1,3,2] | m=11 | *(no flips)* |
| C2 | = [1,2,3,1,2,1] | m=11 | *(no flips)* |
| C3 | = [1,1,2,1,2,3] | m=11 | *(gene 4 flipped P2 → P1)* |
| C4 | = [1,1,1,1,2,1] | m=13 | *(no flips)* |

# Generation 1: New Population

*Elitism keeps the top 2 from generation 0. Fill the rest with the best children.*

| | T1 | T2 | T3 | T4 | T5 | T6 | Makespan | |
|---|---|---|---|---|---|---|---|---|
| y_A | 1 | 1 | 2 | 1 | 2 | 1 | 11 | *elite* |
| y_B | 1 | 2 | 3 | 1 | 3 | 2 | 11 | *elite* |
| C1 | 1 | 1 | 2 | 1 | 3 | 2 | 11 | *child* |
| C3' | 1 | 1 | 2 | 1 | 2 | 3 | 11 | *child* |

**Gen 1 stats**

Best: 11
Worst: 11
Average: 11.0

**Population improvement**

**Average makespan: 12.5 → 11.0.** Every individual now optimal. The population converged in one generation - on this small instance.

# GA vs. SA: Side-By-Side Comparison

| | Simulated Annealing | Genetic Algorithm |
|---|---|---|
| State | single point | population of N points |
| Move | perturb one gene | crossover + mutation |
| Acceptance | Metropolis (T-controlled) | selection (fitness-biased) |
| Diversity from | temperature schedule | population + mutation |
| Hyperparams | T0, alpha, iters | N, p_cx, p_mut, elitism |
| Parallelism | hard (sequential walker) | easy (eval pop in parallel) |

# When to Use Which?

*Both are general. The right pick depends on problem shape and budget.*

## Pick SA when...

- Small-to-medium search space.
- Local moves are cheap; one-walker search is feasible.
- Smooth-ish fitness landscape - few wide barriers.
- Memory tight - SA holds one state.
- You want minimal hyperparameters to tune.

## Pick GA when...

- Search space is huge and rugged.
- Good solutions are made of recombinable pieces (building blocks).
- Multiple objectives - extends naturally to NSGA-II.
- Parallel compute is available - fitness evals are naturally parallel.
- You want a diverse final pool, not just one answer.

# Practical Touches: Elitism and Diversity

*Keep the best. But don't let everyone be the same.*

## Elitism

Carry the top-***k*** individuals from generation ***g*** into generation ***g+1*** unchanged.

**Why:** without it, crossover + mutation can **lose** the best individual found so far.

**Typical:** k = 1-5% of population.

## Diversity preservation

If the population becomes uniform, GA stalls.

**Common techniques:**

- Higher mutation rate when diversity drops.
- Niching / fitness sharing.
- Restart with random injection if stagnant.
- Island models with periodic migration.

# Handling Task-PE Compatibility

*Shrink the gene domain to compatible PEs only.*

## Our example compatibility matrix:

| compat | P1 | P2 | P3 | Domain |
|---|---|---|---|---|
| T1 | 1 | 1 | 1 | {1,2,3} |
| T2 | 1 | 1 | 0 | {1,2} |
| T3 (FFT) | 0 | 0 | 1 | {3} |
| T4 | 1 | 1 | 0 | {1,2} |
| T5 (FFT) | 0 | 0 | 1 | {3} |
| T6 | 1 | 1 | 1 | {1,2,3} |

### Initialization

**y[i] = uniform(domain[i])**
Sample each gene from its compatible PE set, never outside.

### Crossover

Genes are inherited from parents who already have valid values - safe by construction.

### Mutation

**y[i] ← uniform(domain[i] \ {y[i]})**
Sample new value only from this gene's compatible set.

# Next Step Up: Memetic Algorithms

*GA + local search per individual. The current frontier in HLS scheduling research.*

## Memetic Algorithm

```
P = random population
evaluate fitness of each
for each c in P:  c = local_search(c)

while not done:
    sort, keep elite
    while pop not full:
        p1, p2 = select
        c1, c2 = crossover, mutate
        c1 = local_search(c1)
        c2 = local_search(c2)
        add to new pop

return best
```

## Why it works

GA explores globally with crossover. Local search exploits each region thoroughly. Combined, they cover both modes.

## Local search choices

Greedy hill-climb, tabu search, or one round of SA. Cheap variants prevent excess cost.

# Exercises (Genetic Algorithm)

1. **Decode by hand**
   Decode y = [2,1,3,2,1,3] on the Week 11 DAG. Compute the schedule and makespan.

2. **Tournament size**
   Repeat the worked example with k=4 (instead of k=2). How does selection pressure change parent choice?

3. **Two-point crossover**
   Reimplement the worked example using two-point crossover at positions 2 and 4. Compare children to one-point.

4. **Constrained domain**
   Add the Week 11 compat matrix (T3, T5 must run on P3). Initialize a population of 4 and decode.

5. **Mutation sweep**
   Vary p_mut over {0.0, 0.05, 0.1, 0.3, 0.5}. Run 30 generations on a 12-task synthetic DAG. Plot best vs avg.